

GitHub Copilot CLI & SDK

Running the agent outside the editor

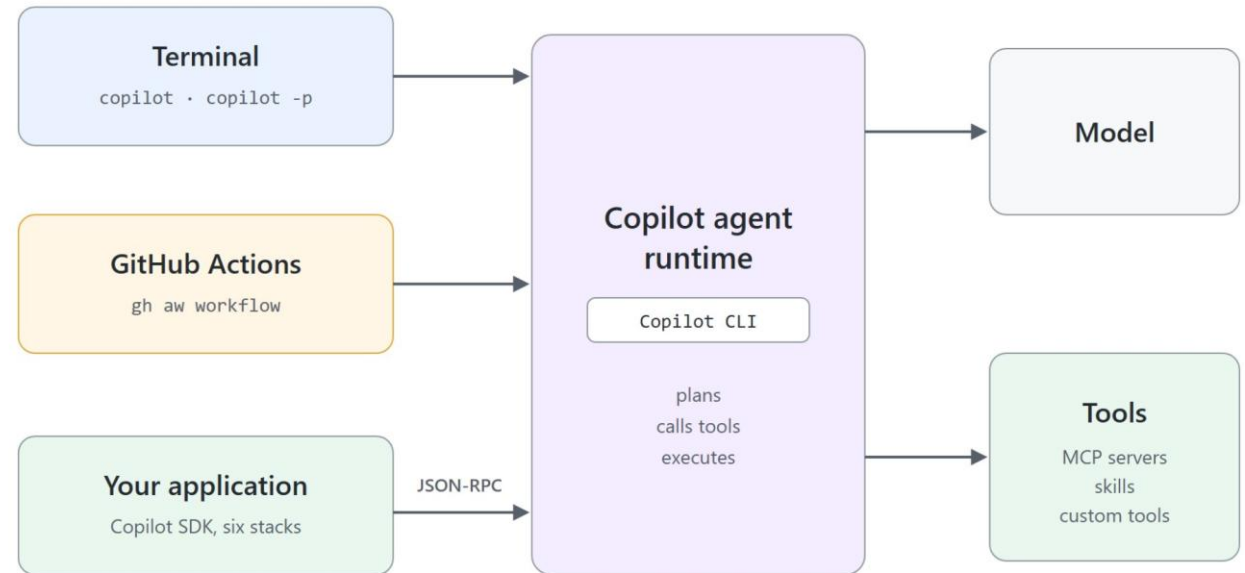
- **CLI:** the agent in your terminal, interactive or headless with `copilot -p`
- **Agentic Workflows:** a prompt that runs on a schedule with `gh aw`
- **SDK:** the same runtime inside your own applications, in six stacks



One runtime, two surfaces

The CLI in your terminal, the SDK in your code

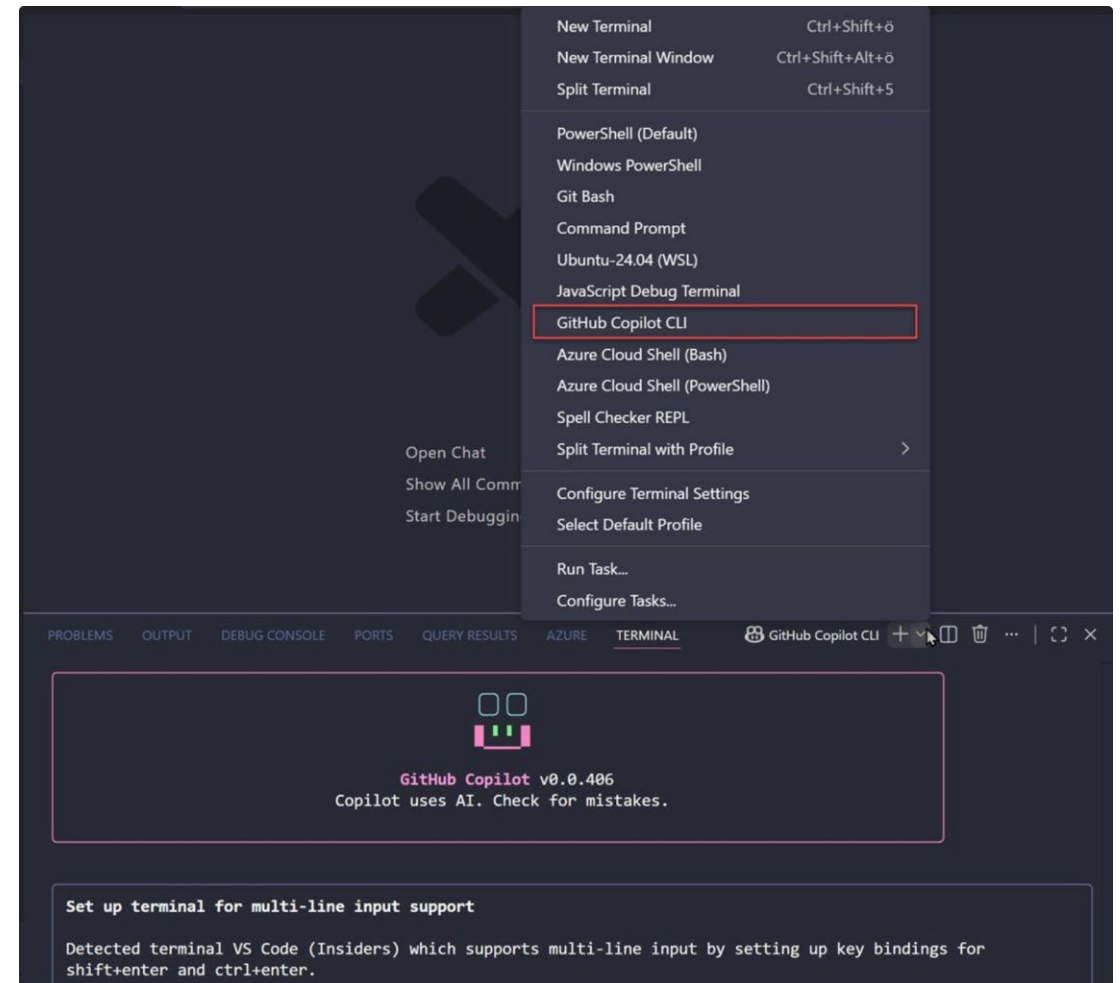
- The **Copilot CLI** drives the agent from an interactive shell or a headless `-p` run
- **GitHub Agentic Workflows** put a CLI-driven agent on an Actions runner
- The **SDK** embeds the same production-tested runtime in your own apps
- Every SDK talks to the CLI over **JSON-RPC** and manages that process for you



The Copilot CLI in your terminal

Install, sign in, ask in plain language

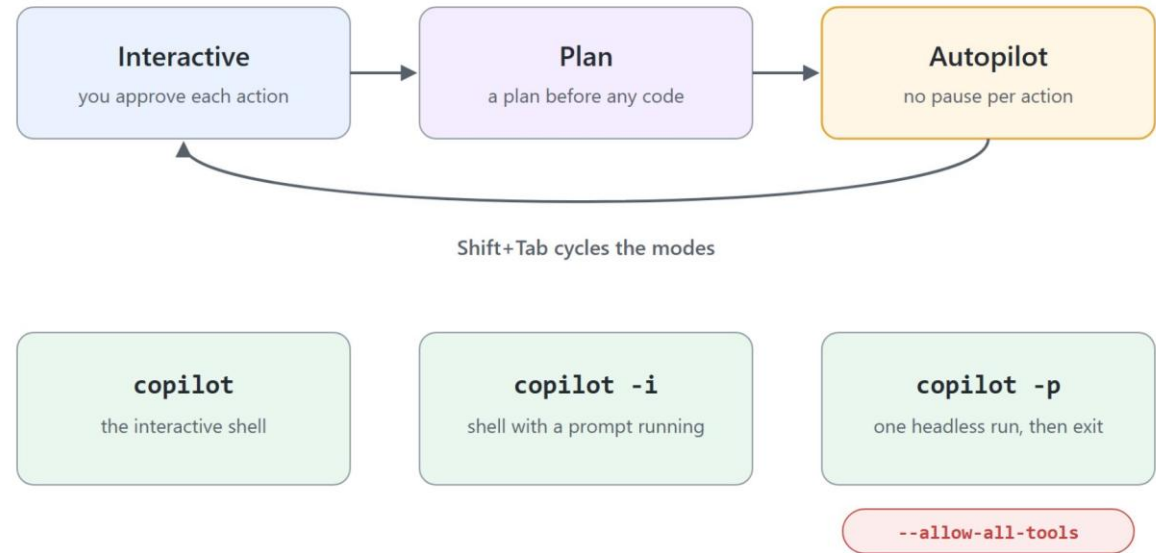
- Install with `npm install -g @github/copilot` or `winget install GitHub.Copilot`
- Start `copilot`, then run `/login` once
- `@` pulls a file into context, `!` runs a shell command yourself
- `/model` picks the model; the list depends on your plan and organization policy



Three modes, three ways to run

Interactive, plan and autopilot; shell, -i and -p

- **Shift+Tab** cycles interactive, plan and autopilot mode
- **Autopilot** works through multi-step tasks without pausing: trusted repositories only
- `copilot -i "<prompt>"` opens the shell with a prompt already running
- `copilot -p "<prompt>" --allow-all-tools` runs headless and exits; add `-s` for scripts

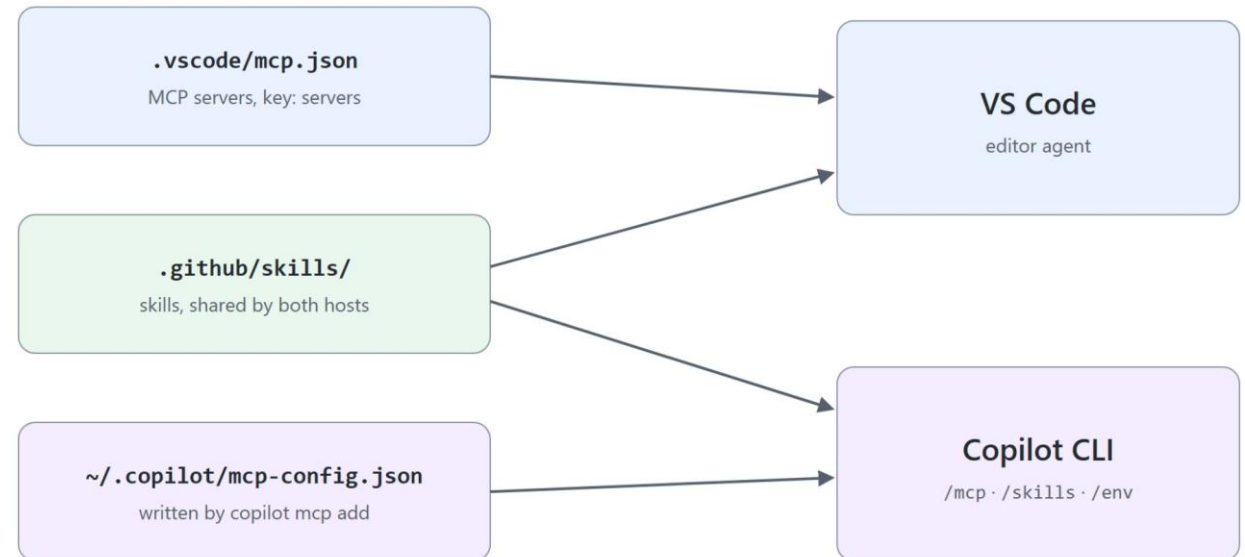


`copilot --continue` or `--resume` picks a session back up

MCP servers and skills in the CLI

Skills are shared by both hosts, MCP registries are not

- `/mcp` in the shell, or `copilot mcp add, list, get, remove` from your normal shell
- The CLI reads `~/.copilot/mcp-config.json`, VS Code reads `.vscode/mcp.json`
- `/skills` or `copilot skill list`: discovered from `.github/skills/` in the working directory
- `/env` shows what actually reached the session; a plugin bundles servers and skills for both hosts

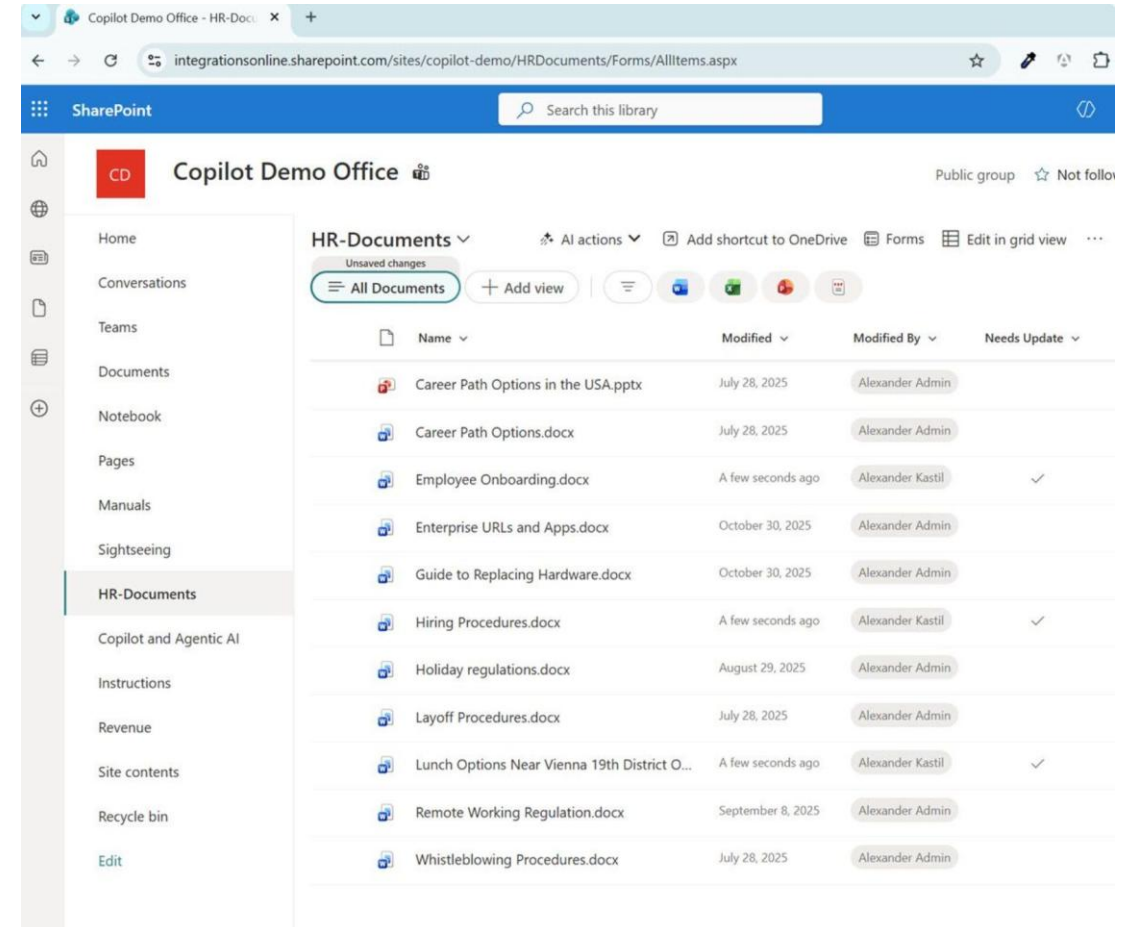


An Agent Plugin carries `skills/` and `mcp.json` to both hosts

HR document updates, automated

A SharePoint library with a Needs Update flag

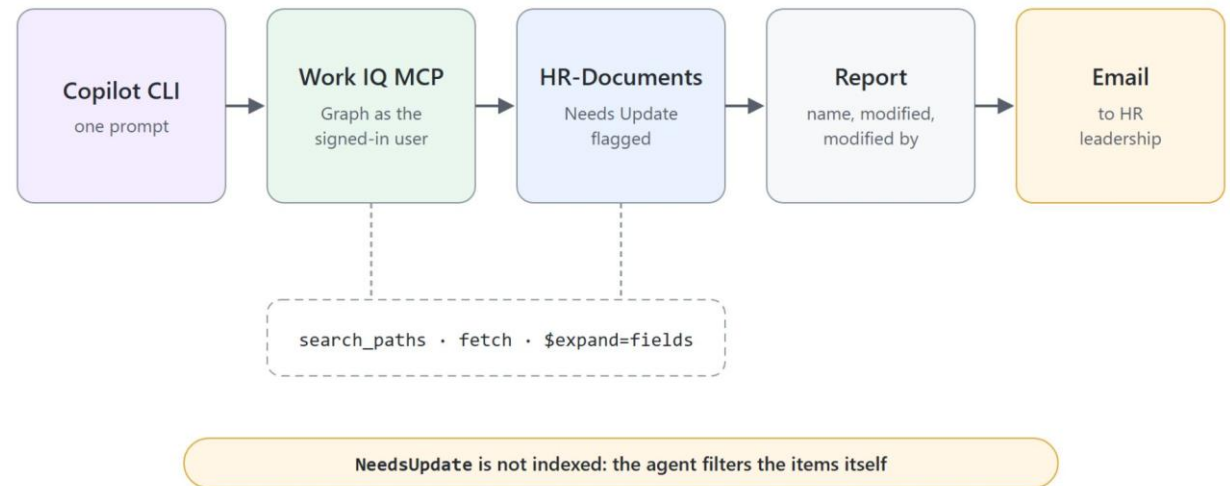
- The HR-Documents library flags stale policies in a **Needs Update** column
- Manual checks are slow; update notifications are delayed or missed
- Copilot CLI queries the library through the **Work IQ MCP server**, as the signed-in user
- It collects name, modified date and modified by, then mails a summary to HR leadership



From the flagged list to an email

What the agent works out through Work IQ

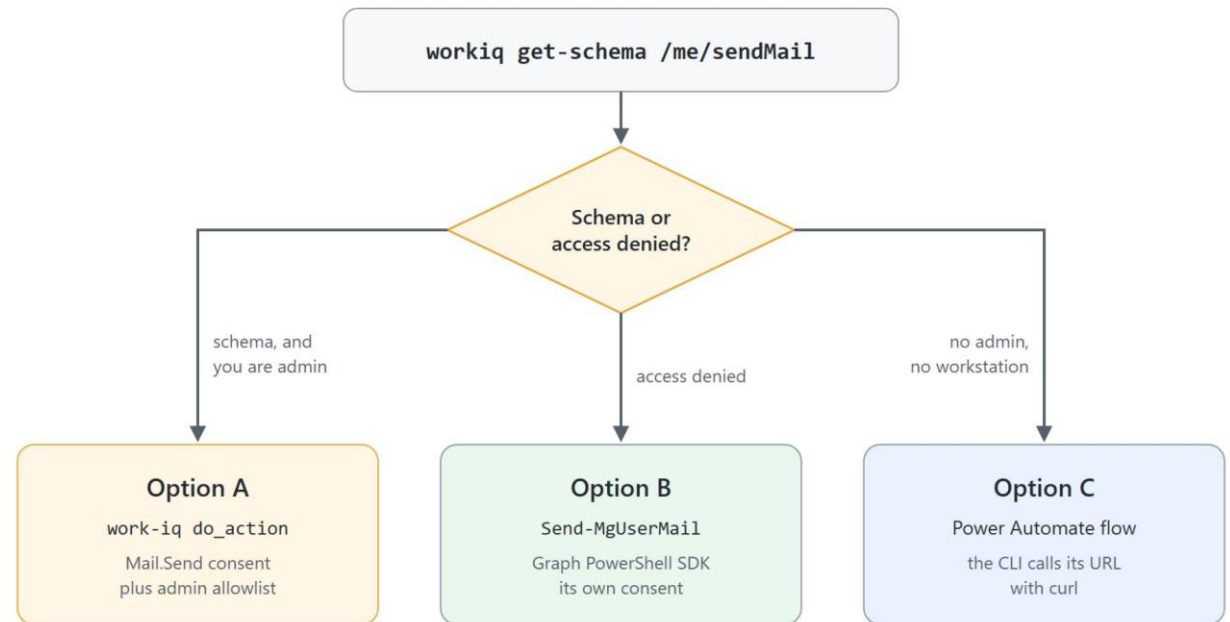
- The agent resolves the site, lists the libraries, reads the columns, fetches items with `$expand=fields`
- `NeedsUpdate` is not indexed, so the agent filters the items itself
- Test the query before the send: **three rows out of twelve** is a correct answer
- Verify the mail in Outlook, not in the transcript



Reading is not sending

Mail.Send is a write scope a read-only Work IQ lacks

- Check first: `workiq get-schema --path /me/sendMail --method post`
- **Option A:** Work IQ sends, after Mail.Send consent and a tenant admin allowlist
- **Option B:** Send-MgUserMail from the Graph PowerShell SDK, with its own consent
- **Option C:** a Power Automate flow the CLI calls with `curl`



Schedule it where the tokens live

A workstation task, not a hosted runner

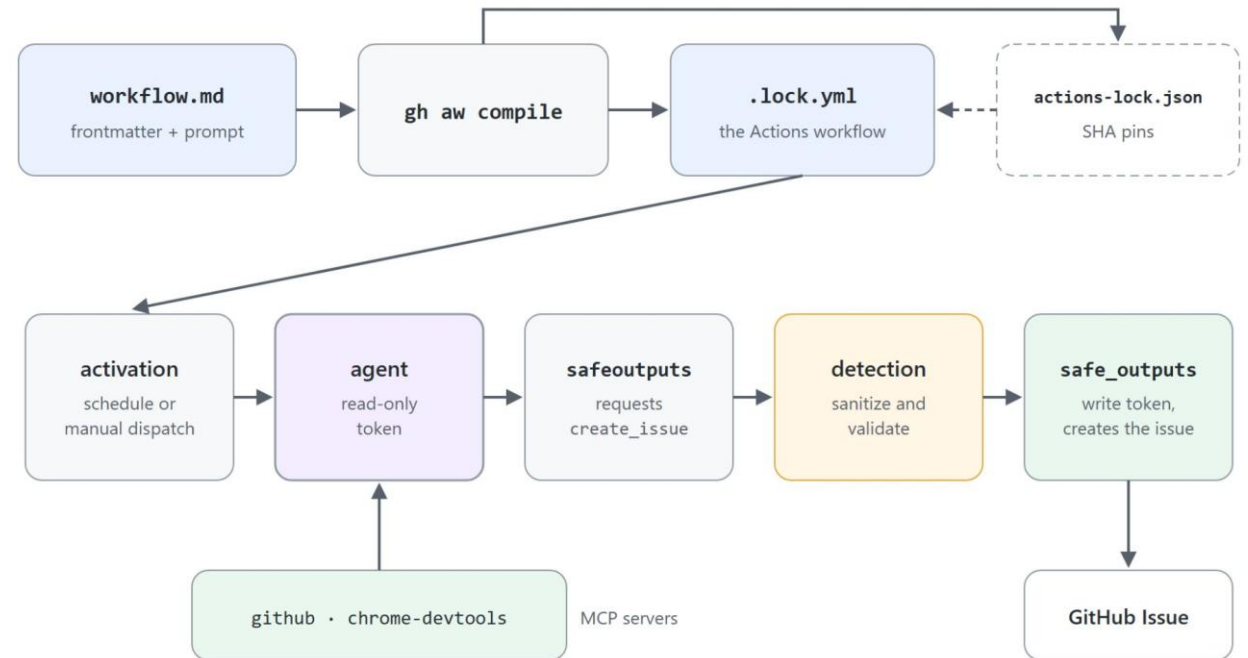
- Unattended runs use `-p` with `--allow-all-tools`, never `-i`
- Task Scheduler runs `update-report.ps1` daily at 8:00
- **Run only if user is logged in:** Work IQ and Graph cache their tokens in your profile
- A GitHub Actions runner has no profile, no token cache and no registered MCP server



GitHub Agentic Workflows

A prompt compiled into an Actions workflow

- A Markdown file: **frontmatter** for trigger, engine, tools and permissions, the prompt below
- `gh aw compile` writes the executable **.lock.yml** and pins actions in `actions-lock.json`
- The agent runs with a **read-only token** and requests writes through `safe-outputs`
- A separate detection job validates the request before a write job creates the issue



Add a workflow with the wizard

gh aw add-wizard opens a pull request

- Install with `gh extension install github/gh-aw` after `gh auth login`
- `gh aw add-wizard githubnext/agentics/daily-repo-status` picks engine and secrets
- The workflow lands on a new branch and a pull request: start from a clean repo
- `gh aw run` triggers it now; run `gh aw upgrade` after every extension upgrade

```
PS D:\git-classes\agentic-sw-engineering> gh aw add-wizard githubnext/agentics/daily-repo-status

Welcome to GitHub Agentic Workflows!

This tool will walk you through adding an automated workflow to your repository.

i This workflow creates daily repo status reports. It gathers recent repository activity (issues, PRs, discussions, releases, code changes) and generates engaging GitHub issues with productivity insights, community highlights, and project recommendations.

✓ Target repository: alexander-kastil/agentic-sw-engineering
✓ Working directory is clean

✓ Selected engine: copilot
✓ Using existing COPILOT_GITHUB_TOKEN secret in repository

The following workflow files will be added:
  • .github/workflows/daily-repo-status.md
  • .github/workflows/daily-repo-status.lock.yml

✓ Created pull request https://github.com/alexander-kastil/agentic-sw-engineering/pull/7
✓ Merged pull request https://github.com/alexander-kastil/agentic-sw-engineering/pull/7

✓ Workflow is ready

X Failed to run workflow: workflow file not found: .github\workflows\daily-repo-status.md

Addition complete!

The workflow 'daily-repo-status' has been added to the repository and will now run automatically.
i This workflow creates daily repo status reports. It gathers recent repository activity (issues, PRs, discussions, releases, code changes) and generates engaging GitHub issues with productivity insights, community highlights, and project recommendations.

Useful commands:
⚡ gh aw status           # Check workflow status
⚡ gh aw run <workflow>   # Trigger a workflow
⚡ gh aw logs             # View workflow logs

Learn more at: https://github.github.com/gh-aw/
```

Ask the codebase

Onboarding as a conversation in the terminal

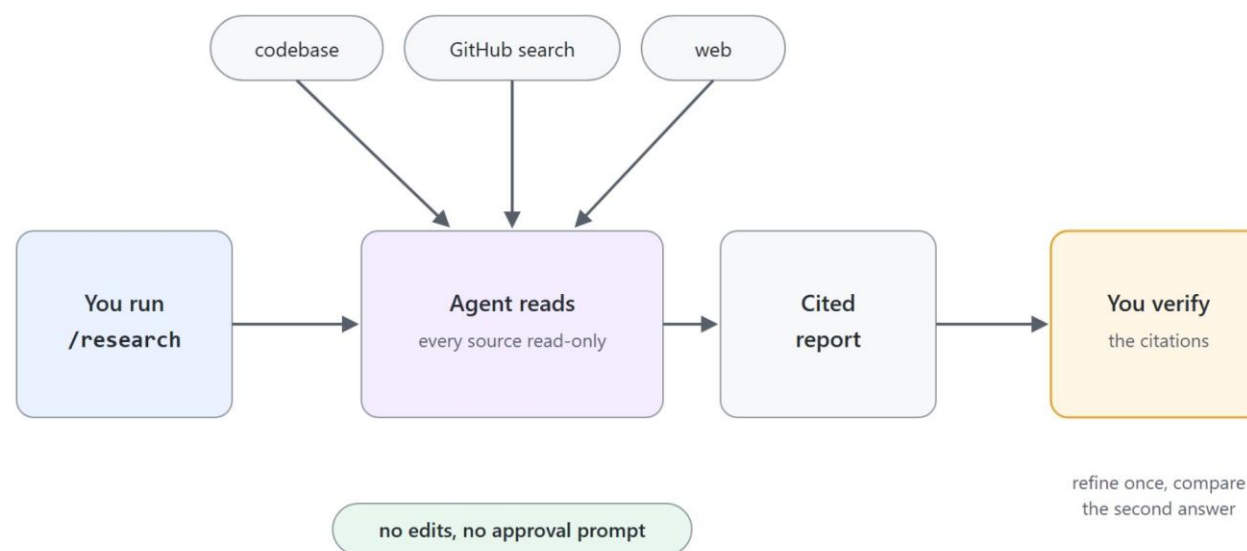
- Ask where a request enters, how auth is enforced, which module owns the logic
- The answer cites **files and lines**: follow them before you trust it
- Ask for an architecture overview or a "start here" guide for one subsystem
- `/share` saves the session; `copilot init` writes an `AGENTS.md` for the next question



/research: read-only and cited

The lowest-risk agent in the course

- Reads across your codebase and GitHub sources; **never writes** to the repository
- No elevated permissions: nothing to approve, nothing to sandbox
- Every claim carries a **citation**: open two or three before you act
- Scope the question so a good answer must name concrete files



`/share` keeps the report as a scoping artifact for an issue or a PR



/rubber-duck: a second opinion

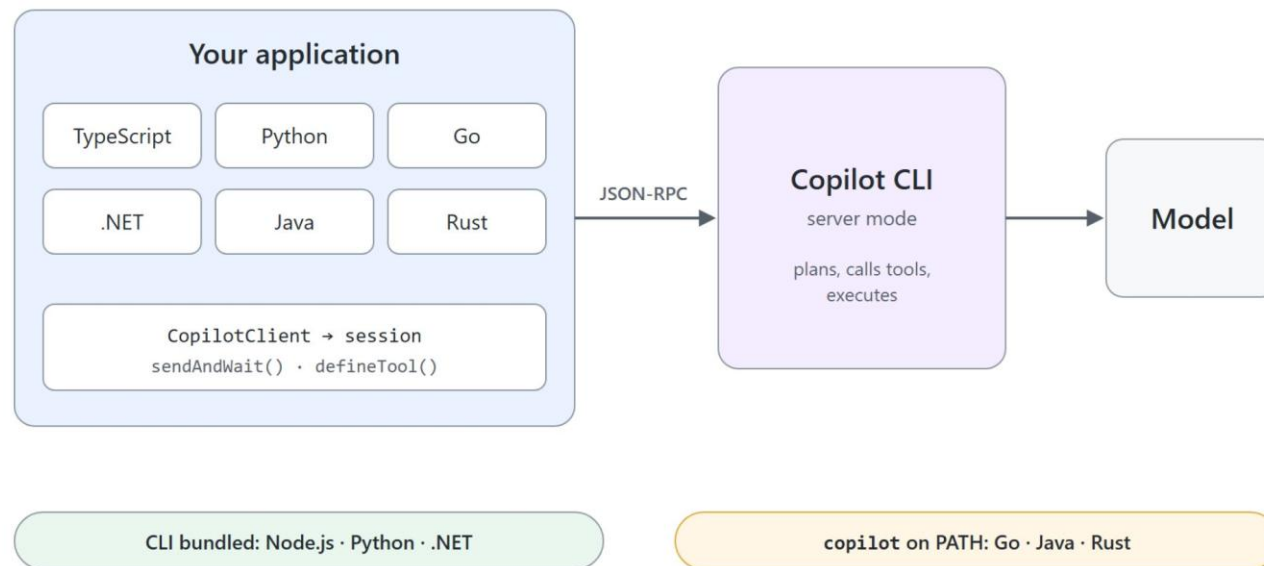
A critic that argues the opposite case

- The agent that wrote the code is the **worst reviewer** of it
- /rubber-duck reads what the session did and reports what was missed
- Review only: **nothing changes on disk**
- Call it after planning and before implementing, where a bad approach is cheapest to kill

The SDK: the agent inside your app

Six stacks on the same runtime

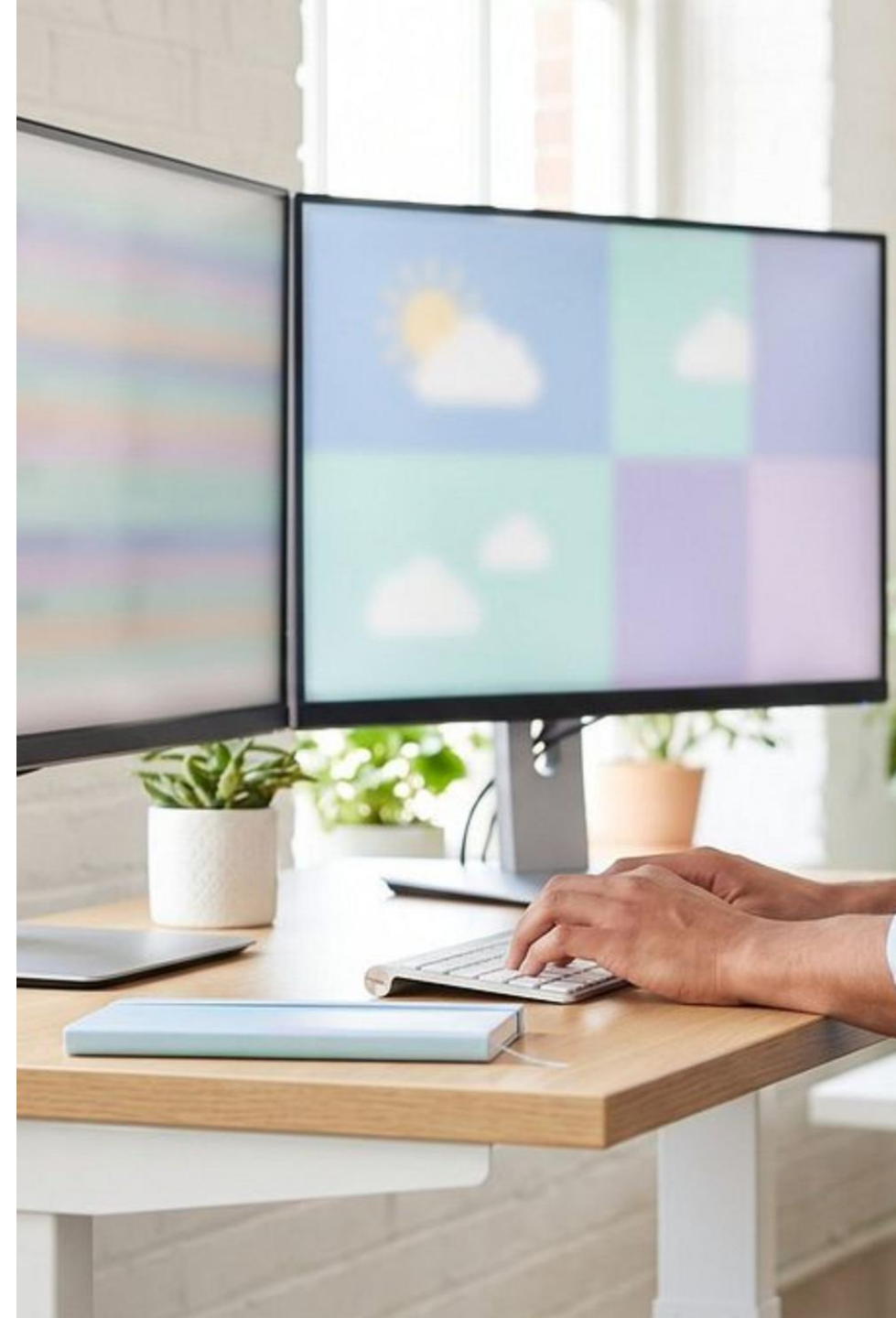
- One first-party package per stack: TypeScript, Python, Go, .NET, Java, Rust
- Node.js, Python and .NET **bundle the CLI**; Go, Java and Rust need `copilot` on PATH
- Create a **CopilotClient**, open a **session**, send with `sendAndWait()` or stream deltas
- `client.listModels()` asks the runtime which models your account may use



Custom tools the agent decides to call

A weather assistant and a code reviewer in TypeScript

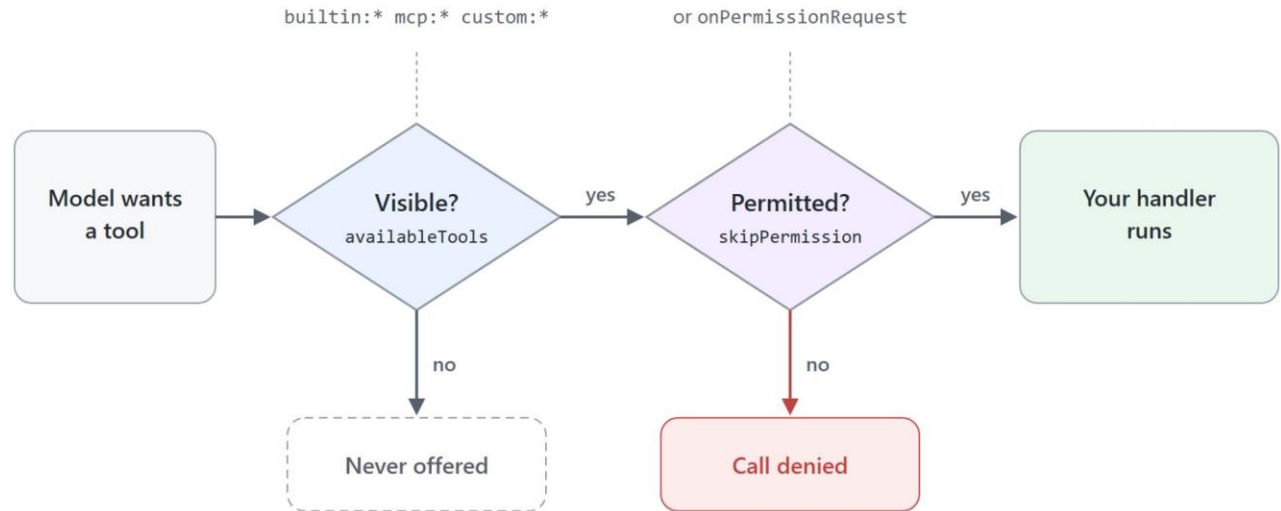
- `defineTool()` takes a name, a description, a parameter schema and a handler
- Copilot decides **when** to call the tool from the user's prompt
- Stream with `assistant.message_delta` and read `event.data.deltaContent`
- The temperatures come from **your handler**, not from the model



Visible, then permitted

Two gates stand in front of every tool call

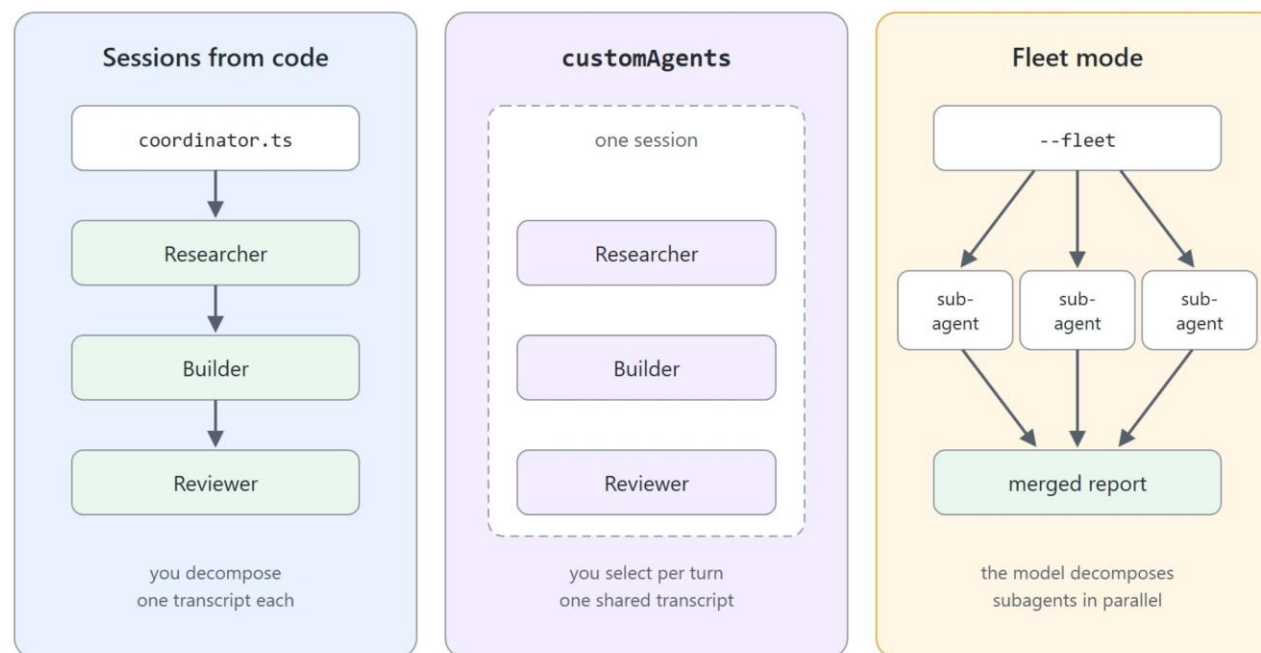
- `availableTools` and `excludedTools` decide which tools the model is **offered**
- Patterns are source-qualified: `builtin:*`, `mcp:*`, `custom:*`
- A session with no `onPermissionRequest` handler **denies** the call
- `skipPermission: true` for handlers that only compute; decide per call otherwise



Three ways to orchestrate specialists

Who decomposes the work, and who sees what

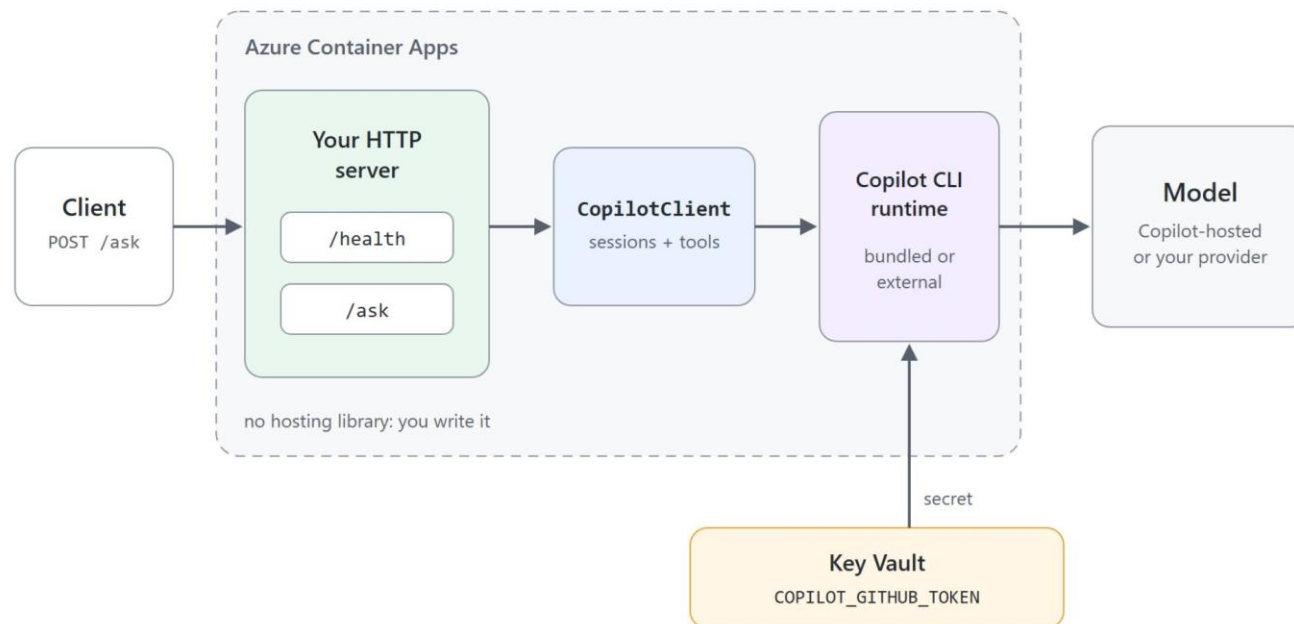
- **Several sessions from your code:** you decompose, each specialist fully isolated
- **customAgents on one session:** select a persona per turn, one shared transcript
- **Fleet mode:** the model decomposes and runs subagents in parallel
- **availableTools:** ["custom:*"] keeps each specialist to the tools you gave it



Hosting an SDK agent on Azure

Your HTTP server, a token from Key Vault

- **No hosting library:** you write the HTTP server, the routes and the health check
- **Bundled** runtime per container, or **external** `copilot --headless --port 4321`
- No `/login` in a container: `COPILOT_GITHUB_TOKEN` as a Key Vault secret
- A `provider` block swaps in Azure OpenAI or Foundry; the agent code does not change



The agent outside the editor

Pick the surface, keep the runtime

- Ask and automate from the terminal: **copilot** and **copilot -p**
- Put a recurring job on a schedule: **gh aw**
- Embed the runtime in your product: the **SDK**
- Check what reached the session: **/env**

